

Multi-Tier Computing

Exam Cheatsheet · All architectures, tiers, middleware, patterns & justifications

1 · ARCHITECTURE STYLES — PICK YOUR TIER MODEL

1-Tier • All components on one machine • Mainframe/standalone • Easy, low-cost, no distributed access	2-Tier • Client ↔ DB Server directly • Fat or thin client • Simple, fast for small teams; poor scalability	3-Tier ★ • Client ↔ App Server ↔ DB • Industry standard for exams • Scalable · Secure · Maintainable	N-Tier • Multiple distributed services • Microservices / SOA style • Max flexibility; complex to build
--	---	---	---

2 · 5-TIER JAVA EE MODEL — USE FOR DETAILED ANSWERS

Client Tier	User interaction, devices, UI presentation	<i>Browser · React/Angular · Android/iOS · Java Applet · Batch process</i>
Presentation Tier	Single sign-on, session mgmt, content creation & delivery	<i>JSP · Servlets · Node.js · PHP · ASP.NET · Nginx</i>
Business Tier	Business logic, transactions, rules, data processing	<i>EJBs · Spring Boot · Django · Express.js · TP Monitor · CICS</i>
Integration Tier	Resource adapters, legacy systems, external APIs, workflow	<i>JMS · JDBC · REST API · ESB · IBM MQ · Kafka · JCA</i>
Resource Tier	Databases, mainframes, B2B systems, external services	<i>MySQL · Oracle · MongoDB · PostgreSQL · Redis · Mainframe</i>

3 · MIDDLEWARE OPTIONS — CHOOSE BASED ON SCENARIO

RPC — Remote Procedure Call • Synchronous · tightly coupled • Masks remote calls as local • Request/reply paradigm • Best for: direct service calls [DCE RPC] [Java RMI] [gRPC]	MOM — Message-Oriented Middleware • Asynchronous · loosely coupled • Messages stored in persistent queues • Client & server run at different times • Best for: high-volume, burst traffic [IBM MQ] [RabbitMQ] [JMS] [Kafka]
ORB — Object Request Broker • Object-oriented · language-independent • Proxy objects for remote calls • Standard interfaces via IDL • Best for: distributed objects [CORBA] [COM/DCOM] [EJBs]	Pub/Sub — Event-Based Middleware • Publishers post events; subscribers filter • Topic-based or content-based filtering • Many-to-many; highly scalable • Best for: real-time events, alerts, fraud [Kafka] [Redis Pub/Sub] [AWS SNS]
ESB — Enterprise Service Bus • Connectivity → Routing → Transform → Orchestrate • Business rules engine + monitoring • Best for: enterprise system integration [Mule ESB] [WSO2] [IBM ESB]	Web Services / REST / SOAP • Language-independent open standard • WSDL = interface description; UDDI = discovery • REST lightweight; SOAP over HTTP/SMTP [REST API] [SOAP] [GraphQL] [gRPC]

4 · TRANSACTION MANAGEMENT — ACID + MODELS + TP MONITORS

ACID Properties • Atomicity — all or nothing • Consistency — leaves system valid • Isolation — concurrent txns don't clash • Durability — survives failures	Transaction Models • Flat — all-or-nothing, short, simple • Chained — durable intermediate commits • Saga — chain + compensating rollback • Nested — subtransactions, recursive • 2-Phase Commit — distributed sync	TP-Lite vs TP-Heavy • TP-Lite: stored procs in DB; single-vendor, fast, limited scale • TP-Heavy: TP Monitor; connection pooling; multi-vendor; enterprise • Use TP-Heavy for banking/large e-commerce	TP Monitor Roles • Process mgmt — start, monitor, load-balance • Transaction mgmt — enforce ACID across resources • Comms — request-reply, queuing, pub-sub • Examples: CICS, Tuxedo, JBoss, WebLogic
--	--	---	--

5 · SCALABILITY STRATEGIES — PEAK TRAFFIC & PERFORMANCE ANSWERS

Vertical Scaling • Add CPU/RAM to existing server • SMP: symmetric multiprocessing • Limit: single machine ceiling	Horizontal Scaling • Add more servers to pool • Load balancer distributes requests • Clusters: shared-disk / shared-nothing	Load Balancing • Round-robin / least-connections • TP Monitor funneling • MOM FIFO or priority queuing
Caching • Redis/Memcached reduce DB hits • CDN for static assets • Cache consistency protocol	DB Replication • Master-slave replication • Read replicas for read-heavy load • Geo-distributed replicas	Partitioning / Sharding • Horizontal sharding by key • Functional decomposition • Removes single-server bottleneck

6 · SECURITY CHECKLIST — APPLY PER TIER

Client Tier	HTTPS/TLS · Input validation · CSRF tokens · XSS prevention
Presentation Tier	Session management · Single sign-on (SSO) · Rate limiting
Business Tier	OAuth/JWT auth · RBAC access control · Firewall · ACL managers
Data Tier	Encryption at rest · DB-level auth · Audit logs · Parameterised queries
Transport Security	SSL/TLS (port 443) · S-HTTP · Kerberos ticket auth · DCE security
Identity & Access	LDAP / Active Directory · JNDI · X.500 directory · Federated identity
Fraud / Anomaly	Rule-based engines in business tier · ML service tier · Kafka + CEP real-time

7 · CLIENT TYPES & SERVER TYPES — NAME YOUR COMPONENTS

Client types:

Non-GUI Client • Batch processes, IoT, sensors • Minimal human interaction • No multitasking needed	GUI Client • Standard windows/web UI • OLTP, business apps • Browser, desktop	OOUI Client • Drag-and-drop, iconic objects • Executive dashboards • Java Beans, ActiveX
Thin Client • All processing on server • Only keystrokes + screen travel • VDI · Citrix ICA · RDP	Fat Client • Heavy local processing • Business logic on client side • Desktop applications	Mobile / Hybrid • 50/50 local + thin-client • Knowledge workers, field users • React Native, PWA

Server types:

Web Server • HTTP/HTTPS, static content [Apache] [Nginx]	App Server • Business logic, EJBs [WebLogic] [JBoss] [Tomcat]	DB Server • SQL queries, ACID [MySQL] [Oracle] [MongoDB] [PostgreSQL]	Transaction Server • ACID across resources [TP Monitor] [CICS] [Tuxedo]
Cache / CDN • Performance layer [Redis] [Memcached] [CloudFront]	Auth Server • Identity, tickets, ACLs [Kerberos] [LDAP] [OAuth/JWT]	File Server • Shared file access [NFS] [DFS] [S3]	Mail / MQ Server • Store-and-forward [SMTP] [IBM MQ] [Kafka]

8 · FAULT TOLERANCE & DCE — HEAVY LOAD / FAILURE SCENARIOS

Clustering • Shared-disk or shared-nothing clusters • Heartbeat 'I'm alive' messages • Failover to alternate node automatically	Redundancy Patterns • Active-active / active-passive servers • Multi-region deployment • Async DB replication	Recovery Mechanisms • ACID transaction rollback • Log-based DFS restart/recovery • Saga compensating transactions • Persistent MOM queue survives restart	
DCE RPC • Client/server calls across platforms • IDL compiler creates stubs • Protocol- & network-independent	DCE Naming • X.500-based distributed directory • Location-independent names • Cells = admin units	DCE Security • Kerberos ticket-based auth • ACL managers per resource • Authentication + encryption	DCE DFS / Threads • Log-based distributed file system • File replication + cache consistency • CMA threads for concurrency

9 · EXAM JUSTIFICATION PHRASES — USE THESE IN YOUR ANSWERS

Performance	• Horizontal scaling + load balancing handles peak traffic • Caching layer (Redis) reduces DB round-trips • TP-Heavy connection pooling limits open DB pipes • CDN serves static assets closer to users
Scalability	• 3-tier separates concerns → each tier scales independently • Stateless app servers → add nodes without session conflict • MOM queues absorb burst traffic asynchronously • DB sharding removes single-server bottleneck
Security	• Clients never access DB directly — middleware enforces this • SSL/TLS encrypts all inter-tier communication • RBAC + ACLs enforce least-privilege access per tier • Firewall + DMZ isolates presentation from business tier
Fault Tolerance	• ACID transactions ensure no partial updates survive failure • Clustered servers with failover eliminate single point of failure • Saga pattern handles long-running distributed transactions • Persistent MOM queues survive server restart and replay

10 · OPEN STANDARDS & ANSWER TEMPLATE

Network / Transport [TCP/IP] [OSI model] [HTTP/HTTPS] [UDP]	OS Standards [UNIX/POSIX] [CORBA IDL] [JNDI] [DCE]	Directory Services [LDAP] [X.500] [Active Directory] [DNS]	DB Interoperability [JDBC] [ODBC] [RDA] [DRDA] [Federated SQL]
Exam Answer Template	1. State tier model (3-tier / 5-tier Java EE) → 2. Name technologies per tier → 3. Explain tier interactions → 4. Address Performance (scaling + caching) → 5. Address Scalability (stateless servers + MOM + sharding) → 6. Address Security (SSL + RBAC + no direct DB access) → 7. For banking: add ACID + 2PC + Saga + clustering → 8. For peak traffic: add load balancing + horizontal scaling + Redis + CDN		